

robot_navigation.ino

Arduino / AVR C – Robot color sensing & line-following navigation

```
1  // --- Global Variables ---
2  // "period" : stores the value of the output wave period (not necessarily in seconds)
3  int period; //microseconds
4  // "timer" : stores the value of TIMER1
5  int timer;
6  // "start_color" : helps indicate the beginning color of where the robot is at. Let 0 = BLUE, 1 = YELLOW
7  int start_color;
8  // "run" : indicates if the robot should move
9  int run;
10 // "lastState" : meant to help indicate falling vs. rising edge of interrupt
11 volatile uint8_t lastState = 0;
12
13 // All of our funtions and ISRs
14 // Drive functions with built-in delay
15 // Robot moves backward
16 void drive_backward_delay(int t){
17     // code to control appropriate pins here:
18     PORTB |= 0b00000110; // set pins 9 and 10 high
19     del(t);              // wait for t milliseconds
20     PORTB &= 0b11111001; // reset pins to low
21 }
22
23 // Robot moves forward
24 void drive_foward_delay(int t){ //454 ms is about 90 degrees
25     PORTB |= 0b00011000; // set pins 11 and 12 high
26     del(t);              // wait for t milliseconds
27     PORTB &= 0b11100111; // reset pins to low
28 }
29
30 // Robot turns left
31 void drive_left_delay(int t){ // 167 ms is about an inch
32     PORTB |= 0b00001010; // set pins 9 and 11 high
33     del(t);              // wait for t milliseconds
34     PORTB &= 0b11110101; // reset pins to low
35 }
36
37 // Robot turns right
38 void drive_right_delay(int t){
39     PORTB |= 0b00010100; // set pins 10 and 12 high
40     del(t);              // wait for t milliseconds
41     PORTB &= 0b11101011; // reset pins to low
42 }
43
44 // Robot stops
45 void stop(int t){
```

```

46     PORTB &= 0b11100001; // reset pins to low
47     del(t);
48 }
49
50 // Our own delay function that will work with variable delays
51 void del(unsigned int t){
52     for(int i = 0; i < t; i++){
53         _delay_ms(1);
54     }
55 }
56
57 // --- ISR: an interrupt function that resets and reads the timer value
58 ISR(PCINT2_vect)
59 {
60     volatile int currentState = PIND & 0b00100000; // reads the state of pin 5
61     // resets the timer to zero on a rising edge,
62     if ((lastState == 0) && (currentState == 32)){
63         TCNT1 = 0;
64     }
65     // and stores the timer value in a variable ("timer") on a falling edge
66     if ((lastState == 32) && (currentState == 0)){
67         timer = TCNT1;
68     }
69     // save pin state so it can be compared during the next interrupt call
70     lastState = currentState;
71 }
72
73
74 // Initialize pins and enable interrupts on pins for color sensor
75 void initColor()
76 {
77     // 1. Set S0, S1, S2, S3 as Outputs
78     DDRC |= 0b00001111; // Set S0-S3 to pins A0-A3 respectivity
79
80     // 2. Set OUT (Pin 5) as Input
81     DDRD &= 0b11011111;
82
83     // 3. Set Frequency Scaling to 20% (S0=H, S1=L)
84     PORTC |= 0b00000001;
85     PORTC &= 0b11111101;
86
87     // 4. Set Filter to Red (S2=L, S3=L)
88     PORTC &= 0b11110011;
89
90     // 5. Initialize Pin Change Interrupt for PCINT21 (Pin 5)

```

```

91     PCICR |= 0b00000100;    // Enable Port D group interrupts
92
93     // 6. Initialize Timer 1
94     // set to normal mode
95     TCCR1A = 0b00000000;
96     TCCR1B = 0b00000000;
97
98     // set prescaler = 1
99     TCCR1B = 0b00000001;
100
101     sei();
102 }
103
104 // Initialize pins for the motors
105 void initMotor()
106 {
107     DDRB |= 0b00011110; // Set pins 11 and 10 to outputs for left motor and pins 12 and 9 for right motor
108     PORTB |= 0b00000000; // set motor pins to 0
109 }
110
111 // Initialize pins for the blackline sensors
112 void initBlackline()
113 {
114     DDRC &= 0b11101111; // Set pin A4 for left blackline sensor
115     DDRD &= 0b10111111; // Set pin 6 for right blackline sensor
116 }
117
118 // --- getColor: calculates the period of the sensor output wave and returns it as a variable
119 int getColor()
120 {
121     // a. Enable the specific bit for Pin Change Interrupt
122     PCMSK2 |= 0b00100000;
123     // b. Add a short delay (5 ~ 10 milliseconds)
124     _delay_ms(50);
125     // c. Disable the specific bit for pin change interrupt
126     // (to prevent further interrupts until getColor is called again)
127     PCMSK2 &= 0b00000000;
128     // d. Return the period in units of time
129     period = (timer * 2) / 16; // Convert the period measurement into microseconds using the fact the one byte is 62.5 ns
130     return period; // return period in ms
131 }
132
133 int main(void){ // setup code that only runs once
134
135     // enable serial port

```

```

136 Serial.begin(9600);
137 del(1000);
138
139
140 // call initColor function
141 initColor();
142
143
144 // call initMotor function
145 initMotor();
146
147
148 // call initBlackline funciton
149 initBlackline();
150
151
152 run = 1; // let robot run in the beginning
153 int start_color = 0; // indicates starting color robot is on. 1 is yellow and 2 is blue. 0 is none
154 int current_color; // indicates current color robot is on. 1 is yellow and 2 is blue. 0 is none
155
156 getColor(); // reads the color the robot is on in the beginning
157 if ((period > 20) && (period < 150)){
158     start_color = 1; // robot is on yellow
159 }
160 else if ((period > 150) && (period < 400)) {
161     start_color = 2; // robot is on blue
162 }
163
164 while (run){
165     getColor(); // reads the color the robot is on while it's moving
166     bool right_black = (PIND & 0b00010000); // right QTI sensor detects black if pin 4 is high
167     bool left_black = (PINC & 0b00010000); // left QTI sensor detects black if pin A4 is high
168
169     // if left QTI sensor detects black --> stop, turn right, and move forward
170     if (left_black && !(right_black)){
171         stop(1000);
172         run = 0;
173         drive_right_delay(600);
174         run = 1;
175         drive_foward_delay(100);
176     }
177
178     // if right QTI sensor detects black --> stop, turn left, and move forward
179     else if (right_black && !(left_black)){
180         stop(1000);

```

```

181     run = 0;
182     drive_left_delay(600);
183     run = 1;
184     drive_foward_delay(100);
185 }
186
187 // if both QTI sensors detect black --> stop, turn right, and move forward
188 else if (left_black && right_black) {
189     stop(1000);
190     run = 0;
191     drive_right_delay(900);
192     run = 1;
193     drive_foward_delay(100);
194 }
195
196 else{
197     // Determine current color robot is on
198     if ((period > 20) && (period < 150)) {
199         current_color = 1; // Robot is on yellow
200     }
201     else if ((period > 150) && (period < 400)) {
202         current_color = 2; // Robot is on blue
203     }
204     else {
205         current_color = 0; // Robot is neither on blue or yellow
206     }
207
208     // Move forward while still on starting color
209     if (current_color == start_color) {
210         drive_foward_delay(100);
211     }
212
213     // Turn right and move forward when color changes to the other one
214     else if ((current_color != start_color) && current_color != 0) {
215         drive_right_delay(600);
216         drive_foward_delay(100);
217         while (run){
218             getColor(); // Robot detects current color it is on
219             bool right_black = (PIND & 0b00010000); // right QTI sensor detects black if pin 4 is high
220             bool left_black = (PINC & 0b00010000); // left QTI sensor detects black if pin A4 is high
221             drive_foward_delay(100); // Robot moves forward
222
223             // if left QTI sensor detects black --> stop, turn right, and move forward
224             if (left_black && !(right_black)){
225                 stop(1000);

```

```
226         run = 0;
227         drive_right_delay(600);
228         run = 1;
229         drive_foward_delay(100);
230     }
231
232     // if right QTI sensor detects black --> stop, turn left, and move forward
233     else if (right_black && !(left_black)){
234         stop(1000);
235         run = 0;
236         drive_left_delay(600);
237         run = 1;
238         drive_foward_delay(100);
239     }
240
241     // if both QTI sensors detect black --> stop, turn right, and move forward
242     if (left_black && right_black) {
243         drive_right_delay(900);
244         drive_foward_delay(100);
245     }
246 }
247 }
248 }
249 }
250 }
```